

Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Отчёт
По лабораторной работе N1
по дисциплине Администрирование систем и сетей
Вариант: -

Работу выполнили:
Молчанов Федор Денисович Р3413
Пышкин Никита Сергеевич Р3413

Работу принял:
Максимов Андрей Николаевич

Оглавление

Цель работы	2
Среда выполнения и реализация стенда	2
Реализованная топология	2
Настройка IPv4-адресации	3
Физические интерфейсы	3
Loopback-интерфейсы	3
Статическая маршрутизация	3
Основные маршруты между loopback-интерфейсами	3
Резервный маршрут и отказ линии	4
Маршрут по умолчанию	5
Итоговая конфигурация и диагностика	6
Адреса интерфейсов	6
Финальные таблицы маршрутизации	6
Ответы на контрольные вопросы	7
Когда статический маршрут добавляется в таблицу IP-маршрутизации?	7
Какой исходный IP будет выбран для ping без явного указания source?	7
Вывод по первой части	7
Часть 2. Маршрутизация OSPF	8
Цель второй части	8
Подготовка FRRouting	8
Исходное состояние перед OSPF	8
Настройка базового OSPF	8
Аутентификация OSPF	9
Анонсирование маршрута по умолчанию	10
Управление выбором маршрута с помощью OSPF cost	10
Асимметричный обратный маршрут	11
Проверка отказоустойчивости OSPF	11
Ответ на контрольный вопрос второй части	12
Вывод по второй части	12

Цель работы

Цель лабораторной работы — получить практические навыки настройки IPv4-адресов, loopback-интерфейсов, статической и динамической маршрутизации. В первой части исследуются статические и резервные маршруты, необходимость обратного маршрута и маршрут по умолчанию. Во второй части на том же стенде настраивается OSPF: формирование соседств, распространение маршрутов, MD5-аутентификация, анонсирование маршрута по умолчанию, управление выбором пути с помощью стоимости и автоматическая перестройка маршрутов при отказе линии.

Исходные лабораторные работы рассчитаны на маршрутизаторы Huawei. В данной реализации устройства воспроизведены тремя Docker-контейнерами с Linux. В первой части использовались стандартные средства ядра Linux (`ip addr`, `ip link`, `ip route`, `ping`, `traceroute`), а во второй части для реализации OSPF применялся FRRouting 8.4.4 (`zebra`, `ospfd`, `vttysh`).

Среда выполнения и реализация стенда

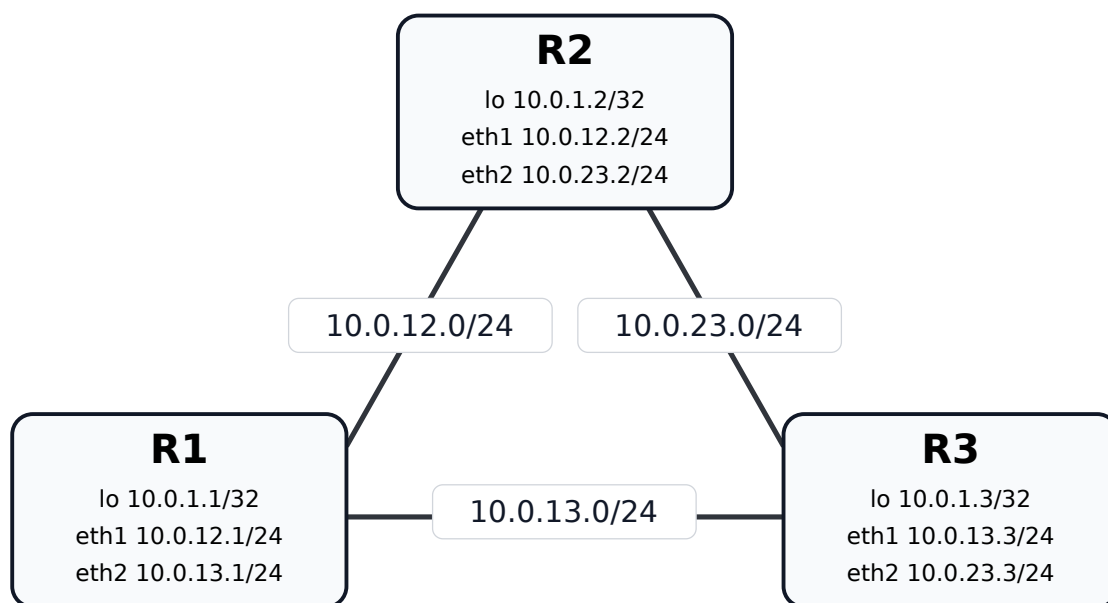
Хостовая система: Arch Linux. Для каждого маршрутизатора создан отдельный Docker-контейнер на базе Ubuntu 24.04. В образ установлены `iproute2`, `iputils-ping`, `traceroute`, `procsps`, `tcpdump`, а для второй части — `frr` и `tini`. Контейнеры запускались без стандартной Docker-сети (`--network none`), с включенной пересылкой IPv4-пакетов (`net.ipv4.ip_forward=1`). Для работы FRRouting контейнерам были предоставлены capabilities `NET_ADMIN`, `NET_RAW` и `SYS_ADMIN`.

Виртуальные линии между маршрутизаторами созданы как пары `veth`. Каждый конец пары перенесен в `network namespace` соответствующего контейнера и переименован в `eth1` или `eth2`.

```
# пример создания одного виртуального линка R1-R2
ip link add r1-r2 type veth peer name r2-r1
ip link set r1-r2 netns <PID_R1>
ip link set r2-r1 netns <PID_R2>
```

Такая реализация позволяет работать с независимыми интерфейсами и таблицами маршрутизации каждого устройства, при этом все контейнеры используют одно ядро Linux хоста.

Реализованная топология



Связи реализованы парами `veth`; R1, R2 и R3 — отдельные Docker network namespace.

Figure 1: Реализованная топология и адресация

Использованы три транзитные сети: 10.0.12.0/24, 10.0.13.0/24, 10.0.23.0/24. Для имитации конечных сетей/клиентов на loopback-интерфейсах используются адреса 10.0.1.1/32, 10.0.1.2/32, 10.0.1.3/32.

Настройка IPv4-адресации

Физические интерфейсы

Настройка адресов выполнялась следующими командами:

```
# R1
ip addr add 10.0.12.1/24 dev eth1
ip addr add 10.0.13.1/24 dev eth2

# R2
ip addr add 10.0.12.2/24 dev eth1
ip addr add 10.0.23.2/24 dev eth2

# R3
ip addr add 10.0.13.3/24 dev eth1
ip addr add 10.0.23.3/24 dev eth2
```

После назначения адресов Linux автоматически добавил маршруты к непосредственно подключенным сетям. Например, на R1:

```
10.0.12.0/24 dev eth1 proto kernel scope link src 10.0.12.1
10.0.13.0/24 dev eth2 proto kernel scope link src 10.0.13.1
```

Связность проверялась по одному разу для каждого физического сегмента. Например, для линии R1–R2 непосредственно после настройки адресов:

```
R1# ping -c 2 -I 10.0.12.1 10.0.12.2
64 bytes from 10.0.12.2: icmp_seq=1 ttl=64 time=0.061 ms
64 bytes from 10.0.12.2: icmp_seq=2 ttl=64 time=0.059 ms
2 packets transmitted, 2 received, 0% packet loss
```

Аналогичная проверка двух остальных непосредственно подключенных сетей также дала 0 % потерь. Повторять одинаковый вывод для каждой линии в отчете нецелесообразно: эта проверка подтверждает только адресацию и состояние канального соединения, но еще не работу статических маршрутов к loopback-адресам.

Loopback-интерфейсы

В Linux использован уже существующий интерфейс lo, которому назначены дополнительные адреса:

```
# R1
ip addr add 10.0.1.1/32 dev lo
# R2
ip addr add 10.0.1.2/32 dev lo
# R3
ip addr add 10.0.1.3/32 dev lo
```

Собственный loopback-адрес помещается Linux в локальную таблицу маршрутизации. Например, на R1 диагностическая команда `ip route show table local` показала:

```
local 10.0.1.1 dev lo proto kernel scope host src 10.0.1.1
```

Статическая маршрутизация

Основные маршруты между loopback-интерфейсами

Для достижения loopback-адресов соседних маршрутизаторов были добавлены статические маршруты:

```
# R1
ip route add 10.0.1.2/32 via 10.0.12.2
ip route add 10.0.1.3/32 via 10.0.13.3

# R2
ip route add 10.0.1.1/32 via 10.0.12.1
ip route add 10.0.1.3/32 via 10.0.23.3

# R3
ip route add 10.0.1.1/32 via 10.0.13.1
ip route add 10.0.1.2/32 via 10.0.23.2
```

Проверка R1–R2 проводилась поэтапно. Сначала маршрут был добавлен только на R1. Команда `ip route get` подтверждает, что R1 уже умеет отправить пакет к R2, однако `ping` не получает ни одного ответа:

```
R1# ip route get 10.0.1.2 from 10.0.1.1
10.0.1.2 from 10.0.1.1 via 10.0.12.2 dev eth1

R1# ping -c 3 -W 1 -I 10.0.1.1 10.0.1.2
3 packets transmitted, 0 received, 100% packet loss

R2# ip route get 10.0.1.1 from 10.0.1.2
RTNETLINK answers: Network is unreachable
```

Причина не в маршруте запроса: echo request доходит до R2, но R2 не знает, куда отправить echo reply с назначением 10.0.1.1. После добавления на R2 обратного маршрута `ip route add 10.0.1.1/32 via 10.0.12.1` повторная проверка сразу стала успешной:

```
R1# ping -c 3 -W 1 -I 10.0.1.1 10.0.1.2
64 bytes from 10.0.1.2: icmp_seq=1 ttl=64 time=0.201 ms
64 bytes from 10.0.1.2: icmp_seq=2 ttl=64 time=0.049 ms
64 bytes from 10.0.1.2: icmp_seq=3 ttl=64 time=0.046 ms
3 packets transmitted, 3 received, 0% packet loss
```

Таким образом, для двустороннего обмена нужны маршрут к адресу назначения и обратный маршрут к адресу источника. После симметричной настройки пар R1–R3 и R2–R3 loopback-интерфейсы всех трех устройств стали взаимно достижимы.

Проверка	Результат	Назначение
R1 10.0.1.1 → R2 10.0.1.2	0 % потерь	проверка маршрутов R1↔R2
R1 10.0.1.1 → R3 10.0.1.3	0 % потерь	проверка маршрутов R1↔R3
R2 10.0.1.2 → R3 10.0.1.3	0 % потерь	проверка маршрутов R2↔R3

Резервный маршрут и отказ линии

Для направления R1↔R2 был создан резервный путь через R3. В исходном оборудовании Huawei для выбора резервного маршрута используется параметр `preference`; в Linux аналогичный выбор среди статических маршрутов был воспроизведен с помощью `metric` — маршрут с меньшим значением выбирается первым.

```
# основной путь
ip route add 10.0.1.2/32 via 10.0.12.2 metric 60 # R1
ip route add 10.0.1.1/32 via 10.0.12.1 metric 60 # R2

# резерв через R3
ip route add 10.0.1.2/32 via 10.0.13.3 metric 100 # R1
ip route add 10.0.1.1/32 via 10.0.23.3 metric 100 # R2
```

До отказа `ip route get` на R1 выбирал прямой next hop, а ответ от соседнего loopback приходил с TTL 64:

```
R1# ip route get 10.0.1.2 from 10.0.1.1
10.0.1.2 from 10.0.1.1 via 10.0.12.2 dev eth1

R1# ping -c 2 -I 10.0.1.1 10.0.1.2
64 bytes from 10.0.1.2: icmp_seq=1 ttl=64 time=0.038 ms
64 bytes from 10.0.1.2: icmp_seq=2 ttl=64 time=0.032 ms
2 packets transmitted, 2 received, 0% packet loss
```

Для моделирования отказа линия R1-R2 была административно отключена с обеих сторон:

```
ip link set eth1 down # R1
ip link set eth1 down # R2
```

После отключения основной маршрут исчез из активной таблицы, и R1 стал использовать резервный next hop 10.0.13.3. Связь между loopback-интерфейсами сохранилась; TTL ответа уменьшился с 64 до 63, поскольку ответ теперь прошел через дополнительный маршрутизатор R3:

```
R1# ip route get 10.0.1.2 from 10.0.1.1
10.0.1.2 from 10.0.1.1 via 10.0.13.3 dev eth2

R1# ping -c 3 -I 10.0.1.1 10.0.1.2
64 bytes from 10.0.1.2: icmp_seq=1 ttl=63 time=0.085 ms
64 bytes from 10.0.1.2: icmp_seq=2 ttl=63 time=0.049 ms
64 bytes from 10.0.1.2: icmp_seq=3 ttl=63 time=0.040 ms
3 packets transmitted, 3 received, 0% packet loss
```

Наиболее наглядная диагностическая команда — traceroute:

```
traceroute to 10.0.1.2 (10.0.1.2)
1  10.0.13.3
2  10.0.1.2
```

Таким образом, после отказа прямого сегмента трафик действительно прошел по пути R1 → R3 → R2.

Отдельно отмечено различие состояний интерфейса Linux: после отключения одного конца veth второй конец оставался административно UP, но имел NO-CARRIER, а маршруты через него отображались с признаком linkdown. После административного down второго конца соответствующие статические маршруты были удалены из таблицы. При последующем ip link set eth1 up Linux восстановил подключенный маршрут 10.0.12.0/24, но не удаленный статический маршрут /32: его потребовалось добавить повторно. Это отличие важно учитывать при моделировании отказа административным отключением интерфейса.

Маршрут по умолчанию

После проверки резервирования линия R1-R2 была включена снова, а прямой обратный маршрут на R2 восстановлен. Специфические маршруты R1 к 10.0.1.2/32 были удалены. До добавления default route диагностическая команда подтвердила отсутствие подходящего пути:

```
R1# ip route get 10.0.1.2 from 10.0.1.1
RTNETLINK answers: Network is unreachable
```

На R1 был добавлен маршрут по умолчанию:

```
ip route add default via 10.0.12.2
```

Итоговая таблица R1:

```
default via 10.0.12.2 dev eth1
10.0.1.3 via 10.0.13.3 dev eth2
```

```
10.0.12.0/24 dev eth1 proto kernel scope link src 10.0.12.1
10.0.13.0/24 dev eth2 proto kernel scope link src 10.0.13.1
```

При этом 10.0.1.2 достигается через маршрут по умолчанию, а для 10.0.1.3 Linux сохраняет более специфичный маршрут /32 через R3. Это демонстрирует правило longest prefix match: наиболее специфичный совпавший префикс имеет приоритет над 0.0.0.0/0.

После настройки default route тот же запрос стал разрешаться через R2:

```
R1# ip route get 10.0.1.2 from 10.0.1.1
10.0.1.2 from 10.0.1.1 via 10.0.12.2 dev eth1

R1# ping -c 2 -I 10.0.1.1 10.0.1.2
64 bytes from 10.0.1.2: icmp_seq=1 ttl=64 time=0.046 ms
64 bytes from 10.0.1.2: icmp_seq=2 ttl=64 time=0.030 ms
2 packets transmitted, 2 received, 0% packet loss

R1# traceroute -n -s 10.0.1.1 10.0.1.2
1  10.0.1.2  0.153 ms
```

Связь R1→R2 восстановилась, а traceroute снова показал прямой путь в один переход.

Итоговая конфигурация и диагностика

Адреса интерфейсов

Устройство	Интерфейс	IPv4-адрес
R1	lo	10.0.1.1/32
	eth1	10.0.12.1/24
	eth2	10.0.13.1/24
R2	lo	10.0.1.2/32
	eth1	10.0.12.2/24
	eth2	10.0.23.2/24
R3	lo	10.0.1.3/32
	eth1	10.0.13.3/24
	eth2	10.0.23.3/24

Финальные таблицы маршрутизации

```
R1
default via 10.0.12.2 dev eth1
10.0.1.3 via 10.0.13.3 dev eth2
10.0.12.0/24 dev eth1 ...
10.0.13.0/24 dev eth2 ...

R2
10.0.1.1 via 10.0.12.1 dev eth1
10.0.1.1 via 10.0.23.3 dev eth2 metric 100
10.0.1.3 via 10.0.23.3 dev eth2
10.0.12.0/24 dev eth1 ...
10.0.23.0/24 dev eth2 ...

R3
10.0.1.1 via 10.0.13.1 dev eth1
10.0.1.2 via 10.0.23.2 dev eth2
10.0.13.0/24 dev eth1 ...
10.0.23.0/24 dev eth2 ...
```

Для диагностики в ходе работы использовались команды `ip -br addr`, `ip -br link`, `ip route`, `ip route show table local`, `ip route get`, `ping` и `tracert`. В отчет включены только результаты, необходимые для подтверждения ключевых этапов, без дублирования однотипного терминального вывода.

Ответы на контрольные вопросы

Когда статический маршрут добавляется в таблицу IP-маршрутизации?

Статический маршрут может быть активным, когда указанный next hop достижим через рабочий интерфейс/существующий маршрут. Если следующий переход недоступен, такой маршрут не может использоваться для пересылки пакетов. В выполненном стенде это наблюдалось при отказе линии: после административного отключения интерфейса маршруты через него переставали участвовать в выборе пути, и использовался резервный маршрут.

Какой исходный IP будет выбран для ping без явного указания source?

Маршрутизатор выбирает исходный адрес в соответствии с выбранным выходным интерфейсом. Для прямого маршрута R1→R2 через сеть 10.0.12.0/24 источником стал бы адрес 10.0.12.1, а не loopback 10.0.1.1. Поэтому для проверки именно связи loopback-to-loopback источник указывался явно (`ping -I 10.0.1.1 ...`; в Huawei — параметр `-a`).

Вывод по первой части

В ходе первой части лабораторной работы была создана трехмаршрутизаторная IP-сеть в Docker-контейнерах без специализированного routing-daemon. Настроены физические и loopback-адреса, статические маршруты, резервный маршрут и маршрут по умолчанию. Экспериментально подтверждены необходимость обратного маршрута, автоматическое переключение на резервный путь при отказе линии и правило выбора наиболее специфичного маршрута. Использование Linux network namespace и veth позволило воспроизвести основные сетевые процессы исходной Huawei-лабораторной стандартными средствами ядра Linux.

Часть 2. Маршрутизация OSPF

Цель второй части

Цель второй части — настроить однозонную динамическую маршрутизацию OSPF на трех маршрутизаторах, проверить формирование соседств и автоматическое появление маршрутов, настроить аутентификацию OSPF, анонсирование маршрута по умолчанию и выбор пути на основании стоимости. Дополнительно проверяется автоматическая перестройка маршрутов после отказа и восстановления канала.

Топология и IPv4-адресация сохранены из первой части: 10.0.12.0/24 соединяет R1 и R2, 10.0.13.0/24 — R1 и R3, 10.0.23.0/24 — R2 и R3. Loopback-адреса: R1 — 10.0.1.1/32, R2 — 10.0.1.2/32, R3 — 10.0.1.3/32.

Подготовка FRRouting

Для реализации OSPF образ маршрутизатора был дополнен FRRouting. В работе использовалась версия FRR 8.4.4. Демон ospfd реализует OSPFv2, а zebra взаимодействует с таблицей маршрутизации ядра Linux и устанавливает рассчитанные маршруты в FIB. Для корректного запуска контейнеров использовались capabilities NET_ADMIN, NET_RAW и SYS_ADMIN.

```
FROM ubuntu:24.04

RUN apt-get update && DEBIAN_FRONTEND=noninteractive apt-get install -y \\\
    iproute2 iputils-ping traceroute procps tcpdump frr tini \\\
    && rm -rf /var/lib/apt/lists/*
```

В контейнерах были запущены только необходимые для данной работы демоны FRR. Проверка на каждом маршрутизаторе показала работающие процессы zebra и ospfd:

```
/usr/lib/frr/zebra -d -A 127.0.0.1 -f /etc/frr/zebra.conf
/usr/lib/frr/ospfd -d -A 127.0.0.1 -f /etc/frr/ospfd.conf
```

Настройка выполнялась параллельно в трех терминалах — по одному для R1, R2 и R3. Это позволило одновременно наблюдать изменение соседств и таблиц маршрутизации на разных устройствах.

Исходное состояние перед OSPF

После создания veth-соединений и назначения адресов непосредственно подключенные интерфейсы были доступны. Например, на R1 проверки двух соседей завершились без потерь:

```
R1# ping -c 2 10.0.12.2
2 packets transmitted, 2 received, 0% packet loss

R1# ping -c 2 10.0.13.3
2 packets transmitted, 2 received, 0% packet loss
```

До запуска OSPF в основной таблице R1 присутствовали только непосредственно подключенные сети:

```
10.0.12.0/24 dev eth1 proto kernel scope link src 10.0.12.1
10.0.13.0/24 dev eth2 proto kernel scope link src 10.0.13.1
```

Поэтому проверка связи между удаленными loopback-интерфейсами была неуспешной:

```
R1# ping -c 2 -W 1 -I 10.0.1.1 10.0.1.2
2 packets transmitted, 0 received, 100% packet loss
```

Таким образом, перед настройкой динамической маршрутизации физическая связность существовала, но маршрутов к удаленным loopback-адресам еще не было.

Настройка базового OSPF

На всех трех устройствах был создан процесс OSPF и явно задан Router ID, совпадающий с loopback-адресом. Все сети были помещены в backbone-area 0.0.0.0.

```
# R1
router ospf
  ospf router-id 10.0.1.1
  network 10.0.1.1/32 area 0.0.0.0
  network 10.0.12.0/24 area 0.0.0.0
  network 10.0.13.0/24 area 0.0.0.0

# R2
router ospf
  ospf router-id 10.0.1.2
  network 10.0.1.2/32 area 0.0.0.0
  network 10.0.12.0/24 area 0.0.0.0
  network 10.0.23.0/24 area 0.0.0.0

# R3
router ospf
  ospf router-id 10.0.1.3
  network 10.0.1.3/32 area 0.0.0.0
  network 10.0.13.0/24 area 0.0.0.0
  network 10.0.23.0/24 area 0.0.0.0
```

После сходимости OSPF маршруты автоматически появились в таблице Linux. На R1:

```
10.0.1.2 via 10.0.12.2 dev eth1 proto ospf metric 20
10.0.1.3 via 10.0.13.3 dev eth2 proto ospf metric 20
10.0.23.0/24 proto ospf metric 20
  nexthop via 10.0.12.2 dev eth1 weight 1
  nexthop via 10.0.13.3 dev eth2 weight 1
```

Для сети 10.0.23.0/24 были установлены два равноценных next hop. Это демонстрирует ECMP: OSPF может одновременно использовать несколько путей с одинаковой суммарной стоимостью.

Повторная проверка loopback-to-loopback после запуска OSPF стала успешной:

```
R1# ping -c 3 -I 10.0.1.1 10.0.1.2
3 packets transmitted, 3 received, 0% packet loss
```

В отличие от первой части, ни один маршрут `ip route add` к удаленному loopback не создавался вручную: его рассчитали ospfd и zebra.

Аутентификация OSPF

На OSPF-интерфейсах была включена криптографическая аутентификация MD5. В качестве идентификатора ключа использовалось значение 1, пароль — H CIA-Datacom. Пример конфигурации R1:

```
interface eth1
  ip ospf authentication message-digest
  ip ospf message-digest-key 1 md5 H CIA-Datacom

interface eth2
  ip ospf authentication message-digest
  ip ospf message-digest-key 1 md5 H CIA-Datacom
```

Аналогичная аутентификация была настроена на соответствующих OSPF-интерфейсах остальных маршрутизаторов. Команда `show ip ospf interface` на R1 подтвердила ее применение на обоих физических интерфейсах:

```
eth1 ... Area 0.0.0.0
  Neighbor Count is 1, Adjacent neighbor count is 1
  Cryptographic authentication enabled
```

```
Algorithm:MD5
```

```
eth2 ... Area 0.0.0.0
Neighbor Count is 1, Adjacent neighbor count is 1
Cryptographic authentication enabled
Algorithm:MD5
```

После согласованной настройки на всех концах OSPF-соседства сохранились в состоянии Full. На R1 наблюдались оба соседа:

Neighbor ID	State	Address	Interface
10.0.1.2	Full/DR	10.0.12.2	eth1:10.0.12.1
10.0.1.3	Full/DR	10.0.13.3	eth2:10.0.13.1

Состояние Full означает, что соседние маршрутизаторы завершили формирование adjacency и синхронизацию базы состояний каналов.

Анонсирование маршрута по умолчанию

R1 использовался как граничный маршрутизатор и был настроен на безусловное анонсирование маршрута 0.0.0.0/0 в OSPF:

```
router ospf
default-information originate always
```

Параметр always позволяет распространять default route даже при отсутствии собственного маршрута по умолчанию в основной таблице R1. Это является функциональным аналогом Huawei-команды default-route-advertise always.

На R2 был получен маршрут по умолчанию через R1:

```
0>* 0.0.0.0/0 [110/1] via 10.0.12.1, eth1

default via 10.0.12.1 dev eth1 proto ospf metric 20
```

На R3 результат аналогичен:

```
0>* 0.0.0.0/0 [110/1] via 10.0.13.1, eth1

default via 10.0.13.1 dev eth1 proto ospf metric 20
```

Таким образом, OSPF-информация, полученная FRR, была не только рассчитана внутри routing-daemon, но и установлена zebra в таблицу маршрутизации Linux.

Управление выбором маршрута с помощью OSPF cost

В FRR исходная стоимость каждого физического OSPF-интерфейса составляла 10. Поэтому прямой путь R1→R2 имел стоимость 10, а альтернативный R1→R3→R2 — стоимость 20. Чтобы принудительно выбрать путь через R3, стоимость eth1 на R1 была увеличена до 30:

```
interface eth1
ip ospf cost 30
```

После пересчета SPF маршрут R1 к loopback R2 изменился:

```
R1# show ip route 10.0.1.2/32
Routing entry for 10.0.1.2/32
Known via "ospf", distance 110, metric 20, best
* 10.0.13.3, via eth2, weight 1
```

```
R1# ip route get 10.0.1.2 from 10.0.1.1
10.0.1.2 from 10.0.1.1 via 10.0.13.3 dev eth2
```

Суммарная стоимость пути через R3 равна $10 + 10 = 20$, что меньше стоимости прямой линии 30. Трассировка подтвердила изменение пути:

```
R1# traceroute -n -s 10.0.1.1 10.0.1.2
1  10.0.13.3
2  10.0.1.2
```

В Linux/FRR конечный R2 ответил на второй hop своим loopback-адресом 10.0.1.2; при этом фактический путь соответствует $R1 \rightarrow R3 \rightarrow R2$.

Асимметричный обратный маршрут

Изменение стоимости выполнялось только на интерфейсе eth1 маршрутизатора R1. Поэтому стоимость направления $R2 \rightarrow R1$ не изменилась. Проверка на R2 показала:

```
R2# ip route get 10.0.1.1 from 10.0.1.2
10.0.1.1 from 10.0.1.2 via 10.0.12.1 dev eth1
```

В результате прямой и обратный пути стали различаться:

```
ICMP request: R1 -> R3 -> R2
ICMP reply:   R2 -> R1
```

Это демонстрирует возможность асимметричной маршрутизации: стоимость OSPF задается для исходящего интерфейса конкретного маршрутизатора и не обязана быть одинаковой в обратном направлении.

Проверка отказоустойчивости OSPF

Для моделирования отказа на R1 был административно отключен интерфейс eth2, соединяющий R1 с R3:

```
ip link set eth2 down
```

После отказа сосед R3 исчез из списка, а соседство R1–R2 осталось в состоянии Full. OSPF автоматически заменил недоступный путь через R3 прямым путем $R1 \rightarrow R2$, несмотря на его более высокую стоимость:

```
R1# show ip ospf neighbor
10.0.1.2 Full/DR 10.0.12.2 eth1:10.0.12.1

R1# show ip route 10.0.1.2/32
Known via "ospf", distance 110, metric 30, best
* 10.0.12.2, via eth1, weight 1

R1# ip route get 10.0.1.2 from 10.0.1.1
10.0.1.2 from 10.0.1.1 via 10.0.12.2 dev eth1
```

Связность при этом сохранилась:

```
R1# ping -c 3 -I 10.0.1.1 10.0.1.2
3 packets transmitted, 3 received, 0% packet loss
```

После восстановления интерфейса:

```
ip link set eth2 up
```

OSPF повторно сформировал соседство с R3. После завершения сходимости оба соседа R1 находились в состоянии Full, а более дешевый маршрут через R3 был возвращен автоматически:

Neighbor ID	State	Address	Interface
10.0.1.2	Full/DR	10.0.12.2	eth1:10.0.12.1
10.0.1.3	Full/DR	10.0.13.3	eth2:10.0.13.1

```

Routing entry for 10.0.1.2/32
  Known via "ospf", distance 110, metric 20, best
    * 10.0.13.3, via eth2, weight 1

```

```

10.0.1.2 from 10.0.1.1 via 10.0.13.3 dev eth2

```

Эксперимент показывает основное преимущество динамической маршрутизации по сравнению со статической: после изменения топологии OSPF самостоятельно пересчитывает SPF и изменяет активный маршрут без ручного добавления нового `ip route`.

Ответ на контрольный вопрос второй части

На шаге изменения стоимости маршрутизатор R2 использует для возврата ICMP-пакетов к loopback R1 прямой маршрут через 10.0.12.1 (eth1). Это подтверждено командой `ip route get 10.0.1.1 from 10.0.1.2`.

Причина заключается в том, что стоимость была увеличена только на исходящем интерфейсе R1→R2. Для R1 прямой путь к R2 стал иметь стоимость 30, поэтому был выбран путь R1→R3→R2 со стоимостью 20. На R2 стоимость собственного прямого интерфейса в сторону R1 не менялась, поэтому обратный ICMP reply передается непосредственно R2→R1. Таким образом, в данном эксперименте сформировалась асимметричная маршрутизация.

Вывод по второй части

Во второй части на том же трехмаршрутизаторном Docker-стенде была реализована однозонная маршрутизация OSPF с помощью FRRouting. После запуска `zebra` и `ospfd` маршрутизаторы автоматически обменялись информацией о сетях и установили маршруты к удаленным loopback-интерфейсам. Была проверена MD5-аутентификация OSPF, распространение маршрута по умолчанию с R1, ECMP для равноценных путей и изменение выбора маршрута через настройку OSPF cost.

Изменение стоимости показало, что OSPF выбирает путь по суммарной метрике, а прямой и обратный маршруты могут различаться. При отключении предпочтительного канала OSPF автоматически переключил трафик на оставшийся путь без потери связности, а после восстановления интерфейса вновь выбрал маршрут с меньшей стоимостью. Таким образом, в отличие от статической маршрутизации первой части, динамический протокол самостоятельно реагирует на изменение топологии и поддерживает актуальные маршруты в таблице Linux.